

Sourcing

In-sourcing, outsourcing

Acquisition

Large systems require time and money to build.

However, many parts of a system can be *generic* enough that it *may* be more efficient to acquire them from a third party.

This is especially true for components that are *not core* to the system's value proposition, but are still necessary for its operation.

- Database management systems
- Authentication services
- Payment processing
- Cloud hosting services

Deciding on whether to build or buy a part of your system is a critical decision that can impact the project's *timeline*, *budget*, and overall success.

Build and buy

Building and buying are ways of defining *how* you are going to acquire a system.

They *generally* have very clear advantages and disadvantages, but *it is* always a case by case basis.

Build

- Full control
- Customization
- Higher cost and risk

Building in house is simply more expensive because it takes time. However it offers you full control of the system, allowing for infinite customizability.

This also relies on the abilities of your developers to *realize* that customizability.

Buying

- Faster and often cheaper
- Less control
- Potential for vendor lock-in

Buying a system or component is often cheaper and faster in exchange for *lower customizability*.

With another main cost being potential vendor lock in.

Build and buy

Some questions you can ask yourself

- Is the capability we're looking for a key *differentiator* for our business
- do we risk yielding *competitive advantage* or *control* by using a third party to acquire it
- how will we need that capability to evolve over the *next three to five years*
- can a third party solution meet our *specific needs*?
- does it offer enough *security*?
- *which vendors* can offer it?
- how long are they likely to *support* it?

Defining your needs

There are generally two categories of *capabilities*

- *Commodity*, and
- *Differentiator*

Commodity capabilities are *required* by your company but aren't required to be built by you. Payment, payroll, auth, etc.

Differentiator capabilities are how your company *differentiates* itself in the market.

Case Study: Algorithms

A logistics company considering if it should build its own route optimization engine or buy one

Case Study: Payment

A small brand is planning on having online orders and payments, should they buy or build their payment services

Case Study: Backend

A startup is looking to create a management website with custom user flows fit for specific businesses, should they build their backend or buy one

In sourcing and outsourcing

In-sourcing refers to the practice of keeping all development and operations *within the organization*, while outsourcing involves contracting *external parties* to handle certain tasks or components.

Both of these approaches *build* the system, but they differ in terms of *who* is doing the work.

In sourcing

In sourcing *guarantees* full control.

- Faster feedback loops and real-time collaboration
- Alignment
- IP Protection
- Long-Term Knowledge Retention

Often, in-sourcing is preferred for anything core to the system, anything strategic, and for any industry that's highly regulated.

Outsourcing

Outsourcing can be *faster* and *cheaper* but with less control and potential risks.

Outsourcing shines when speed, flexibility, and cost optimization matter most.

- Reduced Operational Costs
- Access to Specialized Skills
- Faster Time to Market

Outsourcing is usually done for MVP development, legacy system migrations, and developer tools like QA automation and DevOps support.